

MASTER API PROTOCOL & SYSTEM VERIFICATION SHEET

Environment: XAMPP / Apache / MariaDB / React

Purpose: Technical validation of backend architecture, network security, and API integrity against legacy code defects.

Test ID	Test Category	Target Layer	Expected Master Status	Fail Criteria (Legacy Code)
TEST-01	Web Pipeline & JSON Contract	HTTP / Application Layer	200 OK (Structured JSON)	500 Error / HTML Notice Leak
TEST-02	Preflight CORS Handshake	Web Server / Protocol	204 No Content / 200 OK	405 Method Not Allowed / Missing CORS
TEST-03	SQL Injection Defense	Security / Data Layer	401 Unauthorized	Data Breach (200 OK) / PDO Exception
TEST-04	HTTP Verb Enforcement	Routing / Access Control	405 Method Not Allowed	Unhandled Execution / PHP Warning
TEST-05	Payload Validation	Request Processing	400 Bad Request	500 Server Error / Array Key Warnings

TEST-06	XSS & Input Sanitization	Input Safety / DOM	200 OK (Escaped String)	Live Script Execution (alert) / Unescaped DOM
TEST-07	Unauthenticated Access	Auth / Route Protection	401 Unauthorized / 403	Unauthenticated Data Leak / \$_SESSION Notice

EXECUTIVE SUMMARY & TEST MATRIX1. TEST-01: WEB PIPELINE & ASYNC DATA FLOW

Objective: Verify that the PHP API returns pristine, structured JSON responses directly to React without corrupting the payload with raw HTML notices, warnings, or unhandled exceptions.

Step-by-Step Instructions:

1. Open Google Chrome or Microsoft Edge and navigate to the application frontend.
2. Press **F12** (or right-click and select *Inspect*) to open Developer Tools.
3. Click the **Network** tab from the top navigation bar.
4. Perform an action on the UI (e.g., select active ingredients like *Salicylic Acid* and *Retinol*).
5. Click on the corresponding API request in the Network log (e.g., [clash-rules.php](#)).
6. Inspect the **Response** / **Preview** tab in the right pane.

Expected Master System Output (PASS):

- **Status Code:** 200 OK
- **Content-Type:** application/json
- **Response Body:** Clean, structured JSON object:

JSON

```
{
  "a": "Salicylic Acid",
  "b": "Retinol",
  "message": "Combining Salicylic Acid (BHA) with Retinol...",
  "severity": "Potential Conflict",
  "source": "Dermatology Guidelines"
}
```

```
}
```

Legacy Code Failure Output (FAIL):

- **✗ Status Code:** 500 Internal Server Error or Failed.
- **✗ Response Body:** Raw HTML tags contaminating the output:

HTML

```
<br />
```

```
<b>Warning</b>: Undefined array key "ingredient_ids" in <b>C:\xampp\htdocs\...\</b> on line 12<br />
```

```
<b>Fatal error</b>: Uncaught PDOException: SQLSTATE[42000]...
```

(This HTML pollution breaks React's `res.json()` parser instantly).

2. TEST-02: HTTP HANDSHAKE & PREFLIGHT CORS (curl)

Objective: Verify that the web server properly intercepts and validates browser **OPTIONS** preflight requests before React transmits cross-origin payloads.

Step-by-Step Instructions:

1. Open **Command Prompt**, **PowerShell**, or the **VS Code Terminal**.
2. Execute the following **curl** command against the target backend endpoint:

Bash

```
curl -X OPTIONS -i http://localhost/GlowSem1/glowguard-backend/clash-rules.php
```

Expected Master System Output (PASS):

- **Status Line:** HTTP/1.1 204 No Content (or 200 OK)
- **Headers Received:**

Plaintext

Access-Control-Allow-Origin: *

Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS

Access-Control-Allow-Headers: Content-Type, Authorization

Legacy Code Failure Output (FAIL):

- **✗ HTTP/1.1 405 Method Not Allowed**
- **✗ HTTP/1.1 500 Internal Server Error**
- **✗ Total absence of `Access-Control-Allow-*` header definitions.**

3. TEST-03: SQL INJECTION & PARAMETERIZATION DEFENSE

(REMINDER: ENDPOINT URL CHANGES BASED ON YOUR FILE PATH FOR THE SYSTEM)

Objective: Prove that user inputs are safely parameterized via PDO prepared statements rather than dynamically concatenated into raw SQL strings.

Step-by-Step Instructions:

1. Open **Thunder Client** in VS Code (or launch Postman).
2. Create a **New Request** and set the HTTP method to **POST**.
3. Enter the endpoint URL: <http://localhost/GlowSem1/glowguard-backend/auth/login.php>
4. In the **Headers** tab, set:
 - **Key:** Content-Type
 - **Value:** application/json
5. In the **Body** tab, select **JSON** and paste the malicious payload:

```
JSON
{
  "email": "admin' OR '1'='1",
  "password": "" OR '1'='1"
}
```

6. Click **Send**.

Expected Master System Output (PASS):

- **Status Code:** 401 Unauthorized (or 400 Bad Request)
- **Response Body:** Clean JSON error catching the non-existent literal user:

```
JSON
{
  "status": "error",
  "message": "Incorrect email or password"
}
```

(Confirms the injection string was treated as a safe literal value inside PDO bindings).

Legacy Code Failure Output (FAIL):

- **✗ Bypass Success (200 OK):** Logs in automatically as the first database account without valid credentials.
- **✗ Database Crash (500 Error):** Leaks internal schema details via raw MariaDB syntax exceptions.

4. TEST-04: ROUTE SECURITY & HTTP VERB ENFORCEMENT

Objective: Confirm that endpoints restrict illegal HTTP verbs and reject unsupported request methods cleanly.

Step-by-Step Instructions:

1. Open **Thunder Client**.
2. Set the HTTP method to **DELETE**.
3. Target URL: <http://localhost/GlowSem1/glowguard-backend/auth/login.php>
4. Click **Send**.

Expected Master System Output (PASS):

- **Status Code:** 405 Method Not Allowed
- **Response Body:** Clean JSON routing error:

```
JSON
{
  "status": "error",
  "message": "HTTP Method Not Allowed"
}
```

Legacy Code Failure Output (FAIL):

-  Unhandled execution, empty 200 OK responses, or PHP runtime crashes.

5. TEST-05: PAYLOAD VALIDATION & INPUT SANITIZATION

Objective: Ensure the API validates mandatory array keys and request parameters before initiating database transactions.

Step-by-Step Instructions:

1. Open **Thunder Client**.
2. Set the HTTP method to **POST**.
3. Target URL: <http://localhost/GlowSem1/ glowguard-backend/auth/login.php>
4. Clear the Body payload completely (send {}).
5. Click **Send**.

Expected Master System Output (PASS):

- **Status Code:** 400 Bad Request (or 422 Unprocessable Entity)
- **Response Body:** Structured validation message:

JSON

```
{
  "status": "error",
  "message": "Email and password are required"
}
```

Legacy Code Failure Output (FAIL):

-  500 Internal Server Error with Undefined array key notices spilling into the output stream.

TEST-06: CROSS-SITE SCRIPTING (XSS) & INPUT SANITIZATION

Objective: Verify that malicious script tags embedded in incoming JSON payloads are sanitized or safely escaped by the backend and UI layers rather than executed.

Step-by-Step Instructions:

1. Open **Thunder Client** in VS Code.
2. Set the HTTP method to **POST**.
3. Target an endpoint that accepts text input (e.g., <http://localhost/GlowSem1/glowguard-backend/products.php> or [tracker.php](http://localhost/GlowSem1/glowguard-backend/tracker.php)).
4. In the **Headers** tab, set **Content-Type: application/json**.
5. In the **Body** tab, select **JSON** and paste the XSS payload:

JSON

```
{  
  
  "name": "<script>alert('xss_attack')</script>",  
  
  "category": "Serum"  
}
```

6. Click **Send**.
7. Load the application frontend in Chrome/Edge and view the saved item.

Expected Master System Output (PASS):

- **Status Code:** 200 OK (or 201 Created / 400 Bad Request if HTML is stripped).
- **Frontend Behavior:** React renders the text string literally as `<script>alert('xss_attack')</script>` on screen. **No popup alert executes.**

(Confirms that raw script tags are treated as safe text elements and JSX escaping prevents code execution).

Legacy Code Failure Output (FAIL):

- **✗ Script Execution:** Opening the page in the browser triggers a live JavaScript popup box (**alert**).
- **✗ DOM Injection:** The raw **<script>** tag is rendered directly into the HTML DOM unescaped.

TEST-07: UNAUTHENTICATED ACCESS & ROUTE PROTECTION

Objective: Confirm that protected API endpoints enforce token/session authorization checks and reject unauthenticated requests.

Step-by-Step Instructions:

1. Open **Thunder Client**.
2. Set the HTTP method to **GET** (or **POST**).
3. Target a protected user route:
<http://localhost/GlowSem1/glowguard-backend/auth/profile.php> (or [me.php](#)).
4. Clear all **Authorization** headers or active session tokens.
5. Click **Send**.

Expected Master System Output (PASS):

- **Status Code:** **401 Unauthorized** (or **403 Forbidden**)
- **Response Body:** Structured JSON access denied error:

JSON

```
{  
  
  "status": "error",  
  
  "message": "Unauthorized access. Auth token missing or invalid."  
}
```

Legacy Code Failure Output (FAIL):

- **✗ Data Leak (200 OK):** Returns protected user profile data despite missing auth credentials.
- **✗ Uncaught PHP Notice:** **500 Internal Server Error** spilling **Warning: Undefined variable \$_SESSION** into the response stream.

